Rainier Operations Management System

Technical case study | Morgan Briggs | Finance Manager, Rainier Rapid LLC

Business challenge

Six managers maintained independent spreadsheets and reports, using inconsistent KPI definitions and calculations. The resulting reports contradicted one another, consumed recurring preparation time, and made it difficult for ownership, operations, HR, and dispatch to work from a consistent view.

Excel standardization

I consolidated reporting in Excel, standardized KPI definitions and calculations, and used formulas, VBA, and imports to connect financial statements, revenue, expenses, labor ratios, driver metrics, fleet costs, and period comparisons. This established a common reporting foundation before the newer dashboard.

Access relational foundation

I designed the Access relational schema, table relationships, keys, indexes, and SQL queries supporting employee and driver records, route data, rosters, vehicle assignments, and operational reporting. Rebuilding the earlier Excel VBA roster workflow in Access addressed recurring workbook overwrite and repair problems. Initial roster preparation fell from 60-90 minutes to under three minutes; dispatchers still verify and adjust assignments when final route information arrives.

Current dashboard architecture

SQLite is the primary working database for records and saved history, including payroll, attendance, and roster-related data. Access remains an original, read-only source supported through the Raw Access database refresh import, which preserves the original file and stages its data. JSON supplies canonical inputs and supporting saved data; generated dashboard HTML also carries display data. Normal dashboard use works from SQLite and saved/generated data without requiring a live Access connection.

Automation and analytics

PowerShell automation supports data-processing workflows and Slack messaging. A standardized KPI engine applies common definitions and calculations to dashboard analytics and management reports. Operational and financial views connect workforce and route information with cost analysis, trends, and management decisions. A fully burdened driver-cost model and rolling 13-week cash flow forecast complement this reporting work.

Two distinct import generations

Earlier Access imports used fail-fast behavior: an error stopped processing, records already imported could remain, and a corrected file could be retried with prior partial records replaced. That retry-and-replace workflow belongs to the older Access implementation. Current dashboard imports first stage data, validate it, present issues for user correction, automatically revalidate corrected data, and commit after validation succeeds. The dashboard automatically commits at that point. This staged workflow is distinct from the older partial-record recovery process.

Engineering ownership and AI-assisted development

I own requirements, business rules, workflow design, operational testing, and acceptance. I use AI to accelerate prototyping, framework development, and implementation. I personally review code, debug where I can, test features against real operating scenarios, identify problems, direct corrections, and repeat validation until the feature meets the operational requirement. AI assistance is part of development; responsibility for the requirements and accepted behavior remains mine.

Reporting impact: estimates and assumptions

Estimated organization-wide reporting preparation across six managers fell from at least 48 hours to about 6 hours per week: (48 - 6) / 48 = 87.5%, or at least an 87.5% reduction using 48 as the conservative baseline. My own workload fell from about 20 to about 2 hours weekly: (20 - 2) / 20 = 90%. These are approximate before-and-after workload estimates for comparable recurring reporting preparation, not formal time-study measurements. They assume similar reporting scope and cadence. Personal time is included in the organization-wide total and must not be added again. Savings reflect the combined standardization and automation effort, rather than the dashboard alone; they do not establish payroll savings or eliminated positions.

Implemented work and planned expansion

The local working architecture includes SQLite, supported read-only Access refresh, supporting JSON/HTML data, staged dashboard imports, standardized KPI calculations, dashboard analytics, and PowerShell/Slack automation. Dashboard refinement and operational testing continue. SharePoint, Microsoft Graph, and Microsoft 365 multi-user synchronization are planned expansion work. They are not presented as completed synchronization or an implemented cloud deployment.

Portfolio presentation

Dashboard screenshots will be added after the dashboard is final, using independently generated synthetic data. The existing public interactive demonstration remains a separate fictional illustration, with no employer records or live operational connection.